



UNIVERSITY
OF TRENTO

Department of Industrial Engineering

Git fundamentals for software development

From the basics to the intermediate level

Filippo Faccini

June 16th-17st 2026, Trento

Overview and assessment methods

Introduction

git CLI and GUI

Getting Started with git

Basic usage of git

GitHub

GUI usage

Intermediate usage

Advanced usage

Overview and assessment methods

- ▶ The course is divided 3 slots each of 2 hours.
Thursday 9:00-11:00, 14:00-16:00.
Friday 9:00-11:00.
- ▶ The "exam" Friday 14:00-15:00.
- ▶ For each slot we will have a small break in the middle and at the end for questions.

Exam will be divided into two parts:

- ▶ small homework.
- ▶ trivial online multiple choice test on Google forms.

Failing to complete the homework will result in a failing grade.

- ▶ The homework will take you no more than 10 minutes to complete.
- ▶ The homework will be to create an account and repository on GitHub and push at least 1 file.
- ▶ **You will be asked to provide the link to your repository in the multiple choice exam.**

- ▶ Exam have 30 multiple choice questions, each correct answer is worth 1 point.
- ▶ No points are deducted for incorrect answers.
- ▶ No communication with other candidates is allowed during the exam.
- ▶ You may use notes, books, etc.
- ▶ You can consult the Git User's Manual online (<https://git-scm.com/docs>).
- ▶ You will have a maximum of 1 hour to complete the exam.

Introduction

What is git

Backup vs Versioning



Taking a picture of
your code

Notch Retweeted
Definitely Not Dan @Def_N... · 2h
Replying to @notch
Read your code aloud and put it on
Amazon as an audiobook.



What is git

Backup vs Versioning



Taking a picture of
your code



What people usually do:

- ▶ **Copy** files with different names (i.e., `file_v1.txt`, `file_v2.txt`)
- ▶ **Store** files in different folders (i.e., `backup/`, `archive/`) naming them with dates (i.e., `2026-06-09/`)
- ▶ **Email** files to themselves or others
- ▶ **Cry** when things stop working

What is git



git is a **distributed** version control system (VCS).

A **Version Control System** (VCS) is a system that records changes to a file or set of files over time so that you can recall specific versions later.

Types of VCS:

- ▶ **Local VCS**
- ▶ **Centralized VCS**
- ▶ **Distributed VCS**

What a VCS does:

- ▶ **Backup and versioning**
- ▶ **Collaboration and sharing**
- ▶ **Branching and merging**
- ▶ **Tracking and auditing**

Should we use **git** for everything?

Should we use **git** for everything?

YES!

Should we use **git** for everything?

YES!
BUT...

Should we use **git** for everything?

YES!
BUT...

git is not suited for:

- ▶ **Large** files
- ▶ **Binary** files
- ▶ **Sensitive** files

Born in 2005, **git** is the most popular and widely used VCS in the world.
Other VCSs are:

- ▶ **Subversion (SVN)**
- ▶ **Mercurial (Hg)**
- ▶ **Perforce (P4)**
- ▶ **Team Foundation Server (TFS)**
- ▶ **Concurrent Versions System (CVS)**



- ▶ **Distributed** version control system
- ▶ **Fast** and **efficient**
- ▶ **Secure** and **reliable**
- ▶ **Free** and **open-source**
- ▶ **Scalable** and **flexible**
- ▶ **Popular** and **well-documented**

GitHub is **not** git nor a VCS, but a **web-based** platform for **collaboration** and **sharing** of **git** repositories.

There are several web-based platforms for git repositories:

- ▶ **GitHub** (most popular)
- ▶ **GitLab** (most similar to GitHub)
- ▶ **Bitbucket** (by Atlassian)
- ▶ **Codeberg** (full non-profit Open Source)

git CLI and GUI

git can be used through:

- ▶ **Command Line Interface (CLI)**
- ▶ **Graphical User Interface (GUI)**
- ▶ **Integrated Development Environment (IDE)**

CLI

Pros:

- ▶ Powerful (full control)
- ▶ Flexible (many options)
- ▶ Portable (any terminal, any OS)

Cons:

- ▶ You will need to learn it
- ▶ It is not user-friendly
- ▶ It is not visual

GUI

Pros:

- ▶ It is user-friendly
- ▶ It is visual
- ▶ It is easier to learn

Cons:

- ▶ Have less control
- ▶ Have several limitations
- ▶ Are less powerful

Take home message: Use the right tool for the right job.

- ▶ **CLI** for:
 - ▶ Advanced users
 - ▶ Power users
 - ▶ Developers
 - ▶ Remote operations
- ▶ **GUI** for:
 - ▶ Beginners
 - ▶ Casual users
 - ▶ Designers

- ▶ **SourceTree** (Atlassian) only for Mac and Windows (free)
- ▶ **GitHub Desktop** (GitHub) only for Mac and Windows (free)
- ▶ **GitKraken** (Axosoft) for Mac, Windows, and Linux
- ▶ **IDEs extensions**

Getting Started with git

- ▶ **Linux:** `sudo apt-get install git`
- ▶ **Mac:** `brew install git`
- ▶ **Windows:** download latest version of git from <https://git-scm.com/downloads>

Configure git with your name and email address.

```
1 git config --global user.name "John Doe"  
2 git config --global user.email "john.doe@unitn.it"
```

Get help on git configuration.

```
1 git config --help  
2 git config -h
```

Basic usage of git

Create first repository



25

Create a new directory and initialize a git repository.

```
1 mkdir git-course
```

Navigate (Change directory -> cd) into the new directory.

```
1 cd git-course
```

Initialize a git repository.

```
1 git init
```

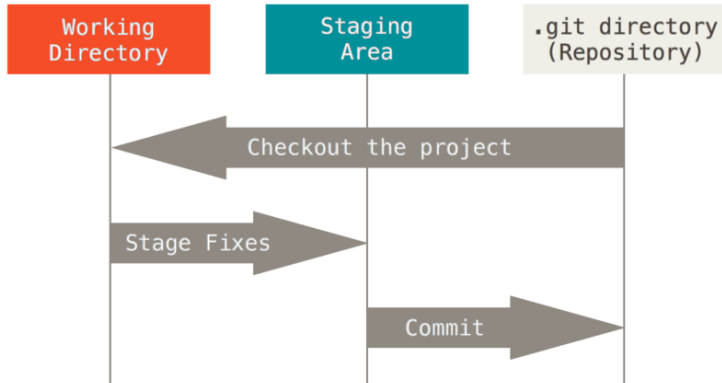
Where are all the information stored



Everything is stored in the `.git` directory.

```
1 ls -a
```

To see hidden files and directories use the `-a` option (all).
Do not touch the `.git` directory unless you know what you are doing.
If you brake it, you will most likely lose all the history of your project.



Inside the `git-course` directory create a new file.

```
1 echo "# test" >> README.md
```

Add another text txt file.

```
1 echo "Hello World" >> hello.txt
```

Add the files to the staging area.

```
1 git add README.md hello.txt
```

Commit the changes.

```
1 git commit -m "feat: add initial project files"
```

Warning: Always add a meaningful message to your commits.

Note: Once committed the staging area is not empty!

Example

2/3



Let's modify the README .md file.

```
1 echo "## test" >> README.md
```

Add the file to the staging area.

```
1 git add README.md
```

Commit the changes.

```
1 git commit -m "fix: correct README content"
```

Delete the `hello.txt` file.

```
1 rm hello.txt
```

Add the file to the staging area.

```
1 git add hello.txt
```

Commit the changes.

```
1 git commit -m "chore: remove hello.txt"
```

Note: you add the file even if you want to delete it.

Each commit has a unique identifier (ID generated).
To see the commit history use the following command.

```
1 git log
```

Git do not store the deltas but the snapshots, and it is very efficient in storing files.
We will see how to use this command later.

1. Modify the README.md file.
2. Check the status of the repository.
3. Add all the files to the staging area.
4. Check the status again.

```
1 echo "## test" >> README.md
2 git status
3 git add .
4 git status
```

Skip the staging area



Modify the README.md file.

```
1 echo "## test" >> README.md
```

Commit the changes adding everything in the staging area.

```
1 git commit -a -m "fix: correct README content"
```

The `-a` option adds all the modified files to the staging area.
A compact version of the above command is:

```
1 git commit -am "fix: correct README content"
```

**Take home message is:
Commit often, commit early, commit with
meaningful messages.**

Golden rules:

- ▶ Not at every keystroke, but when you reach a milestone.
- ▶ Not after a week of work.
- ▶ Do not mix different changes in the same commit.
- ▶ Do not be cryptic, be clear and concise. (e.g. "Fix bug" may look good now, but in a month you will not remember what bug you fixed)

Example

Commits on Mar 27, 2026		
fix: compilation with submodules	faeb354	 
 Pippo98 committed on Mar 27		
fix: change from fetchcontent to git submodules	49a9f8b	 
 Pippo98 committed on Mar 27		
feat: add clang-format, formatted code, update copyright	7d7d29f	 
 Pippo98 committed on Mar 27		
merge: dev	364750a	 
 Pippo98 committed on Mar 27		
docs: update doxygen	491a05c	 
 Pippo98 committed on Mar 27		
fix: initialize data	1b3546f	 
 Pippo98 committed on Mar 27		
Commits on Mar 1, 2026		
fix: remove all warnings on windows too	2eeb6cd	 
 Pippo98 committed on Mar 1		
fix: temporary remove narrowing conversion	9568b51	 
 Pippo98 committed on Mar 1		

Remove a file from the repository and the working directory.

```
1 git rm file.txt
```

Remove a file from the repository but keep it in the working directory.

```
1 git rm --cached file.txt
```

Remove a directory from the repository and the working directory.

```
1 git rm -r directory
```

Remove a directory from the repository but keep it in the working directory.

```
1 git rm -r --cached directory
```

Remove a file from the repository and the working directory.

```
1 rm file.txt
2 git add file.txt
3 git commit -m "chore: remove file.txt"
```

Remove a file from the repository but keep it in the working directory.

```
1 git rm --cached file.txt
2 git commit -m "chore: remove file.txt"
```

Remove a directory from the repository and the working directory.

```
1 rm -r directory
2 git add directory
3 git commit -m "chore: remove directory"
```

Remove a directory from the repository but keep it in the working directory.

```
1 git rm -r --cached directory
2 git commit -m "chore: remove directory"
```

Rename a file in the repository and the working directory.

```
1 git mv old.txt new.txt
```

Rename a file in the repository but keep it in the working directory.

```
1 git mv old.txt new.txt  
2 git rm --cached old.txt
```

Undo the last commit



Undo the last commit but keep changes staged.

```
1 git reset --soft HEAD~1
```

Undo the last commit and unstage changes (keep files modified).

```
1 git reset --mixed HEAD~1
```

Undo the last commit and discard all changes (destructive).

```
1 git reset --hard HEAD~1
```

Warning: `--hard` cannot be undone!

Amend the last commit



Add forgotten changes to the last commit.

```
1 git add forgotten_file.txt
2 git commit --amend --no-edit
```

Fix the commit message.

```
1 git commit --amend -m "fix: update README content"
```

Warning: Only amend commits that have not been pushed to a shared repository.

Undo a specific commit by creating a new commit that reverses it.

```
1 git revert commitID
```

This is safe for shared repositories; it creates a new commit instead of rewriting history.

Difference from `git reset`:

- ▶ `git reset`: rewrites history (use only locally)
- ▶ `git revert`: creates new commit (safe for shared repos)

Create a .gitignore file in the repository root.

```
1 echo "file.txt" >> .gitignore
2 echo "directory/" >> .gitignore
3 git add .gitignore && git commit -m "chore: add .gitignore"
```

Note: .gitignore only works for untracked files.

If you accidentally committed a file that should be ignored:

```
1 git rm --cached file.txt
2 echo "file.txt" >> .gitignore
3 git commit -m "chore: remove file.txt and add to .gitignore"
```

Templates available at <https://github.com/github/gitignore>

How to know which file to ignore



Most of the time it is clear which files to ignore. However, as a rule of thumb, you should ignore:

- ▶ Compiled files.
- ▶ Log files.
- ▶ Temporary files.
- ▶ Configuration files.
- ▶ Files generated by the system.
- ▶ Files generated by the IDE.
- ▶ Pdf files (that are not the final version).
- ▶ Files that contain sensitive information.

You can find templates for the `.gitignore` file at <https://github.com/github/gitignore>.

The `git status` command shows the status of the repository.

M means modified, A means added, D means deleted, ?? means untracked.

MM means modified and staged, AM means added and modified, MD means modified and deleted.

A `file.txt` means the file is staged, M `file.txt` means the file is modified.

D `file.txt` means the file is deleted, ?? `file.txt` means the file is untracked.

MM `file.txt` means the file is modified and staged, AM `file.txt` means the file is added and modified.

MD `file.txt` means the file is modified and deleted.

If you staged a file by mistake, unstage it.

```
1 git restore --staged file.txt
```

Unstage all files.

```
1 git restore --staged .
```

Legacy: `git reset HEAD file.txt`

Discard changes in working directory (restore from staging area).

```
1 git restore file.txt
```

Discard all changes.

```
1 git restore .
```

Restore a file from a specific commit.

```
1 git restore --source=commitID file.txt
```

Legacy: `git checkout - file.txt`

Show changes in working directory (not staged).

```
1 git diff
```

Show changes between staging area and last commit.

```
1 git diff --staged
```

Show changes between working directory and last commit.

```
1 git diff HEAD
```

Use a visual tool instead of terminal output.

```
1 git difftool          # configure with: git config --global diff.tool meld
```

The `git log` command shows the commit history of the repository.

```
1 git log
```

The `git log` command shows the commit history of the repository with a graphical representation.

```
1 git log --graph
```

The `git log` command shows the commit history of the repository with a graphical representation and the commit history of the branches.

```
1 git log --graph --oneline --all
```

If you want to see the history in reverse order use the following command.

```
1 git log --reverse
```

If you want to see the commit history of a specific file use the following command.

```
1 git log -- file.txt
```

Show a specific commit



The `git show` command shows the changes of a specific commit.

```
1 git show commitID
```

`git ls-tree` command shows the three of a specific commit.

```
1 git ls-tree commitID
```

Files are called blobs while directories are called trees.

If you are using git version 2.23 or later, you can use the restore command.

```
1 git restore --staged file.txt
2 git restore file.txt
```

If you want to unstage all the files and discard the changes use the following command.

```
1 git restore --staged .
2 git restore .
```

You can also restore files from a specific commit.

```
1 git restore --source commitID file.txt
```

Create a new directory and initialize a git repository.

```
1 mkdir mySecondRepo
2 cd mySecondRepo
3 git init
```

Create a new file and add it to the staging area.

```
1 echo "# test" >> README.md
2 git add README.md
```

Commit the changes with a meaningful message.

```
1 git commit -m "feat: add README"
```

Add the remote repository and push the changes.

```
1 git remote add origin path
2 git push origin main
```

Difference from `git push` and `git push -u`



52

The `git push` command pushes the changes to the remote repository.

```
1 git push origin main
```

The `git push -u` command pushes the changes to the remote repository and sets the upstream branch.

```
1 git push -u origin main
```

The upstream branch is the default branch for the `git pull` and `git push` commands.

Why this cumbersome procedure?

Can't I just copy and paste the files?



The `git push` command is more than just copying and pasting the files. It is a distributed version control system that allows you to collaborate with other people.

It keeps track of the changes, who made them, when they were made, and why they were made.

It allows you to work on different features at the same time without conflicts.

It allows you to revert to a previous state of the project.

It allows you to merge different branches of the project.

Moreover, now we will see how to store the changes in the cloud.

GitHub

What is GitHub



- ▶ GitHub is a web-based platform for version control using git.
- ▶ It is widely used for software development.
- ▶ It provides a lot of features for collaboration.

Create an account

- ▶ Go to <https://github.com>
- ▶ Create an account.
- ▶ Verify your email address.
- ▶ Set up your profile.
- ▶ Create a repository called **git-course-2026**.
- ▶ ...

What are SSH keys?

ssh keys are a pair of cryptographic keys that can be used to securely connect to a remote server.

Why do we need SSH keys?

- ▶ They are more secure than passwords.
- ▶ They are more convenient than passwords.
- ▶ They are required to connect to GitHub.

We need to:

- ▶ Generate a new SSH key.
- ▶ Add the SSH key to your GitHub account.

```
1 ssh-keygen -t rsa -b 4096 -C "id_rsa.pub"  
2 cat ~/.ssh/id_rsa.pub
```

Copy the output and paste it in your GitHub account in the SSH keys section of the settings.

Create your first repository



- ▶ Create a new repository on GitHub.
- ▶ Initialize a git repository in your local directory.
- ▶ Add the remote repository to your local repository.
- ▶ Push your local repository to the remote repository.

```
1 echo "# test" >> README.md
2 git init
3 git add README.md
4 git commit -m "chore: initial commit"
5 git branch -M main
6 git remote add origin git@github.com:filippofaccini/git-course-2026.git
7 git push -u origin main
```

Alternatively, you can use the following commands:

```
1 git remote add origin https://github.com/filippofaccini/git-course-2026.git
2 git branch -M main
3 git push -u origin main
```

Clone a repository



- ▶ Clone a repository from GitHub.
- ▶ Change the directory to the cloned repository.

```
1 git clone https://github.com/filippofaccini/git-course-2026.git
```

- ▶ Pull changes from the remote repository.

```
1 git pull
```

pull is equivalent to fetch and merge. (we will see this later)

▶ Push changes to the remote repository.

```
1 git push
```

You can only push changes if you have write access to the repository.

GUI usage

In this section, we will use **SourceTree** and **Visual Studio Code** to show how to use a graphical user interface (GUI) to interact with git repositories. Why **SourceTree** and **Visual Studio Code**? Because they are:

- ▶ **Free and open-source**
- ▶ **Cross-platform**
- ▶ **User-friendly**
- ▶ **Powerful and flexible**
- ▶ **Well-documented**

We initialize a new git repository using **SourceTree** and **Visual Studio Code**.
Can be done also for already existing repositories.

We will have a visual comprehension of the git repository structure and how it is easier to understand the changes made to the files, stage and unstage them, commit them, and push them to the remote repository.

We will get to the point where GUIs are not enough and we will need to use the command line interface (CLI) to solve some issues.

Intermediate usage

A branch is a parallel version of a repository starting from a specific commit.
Can be used to develop new features, fix bugs, or experiment with new ideas without affecting the main branch (our stable code).
Branches are lightweight and easy to create.

Branches

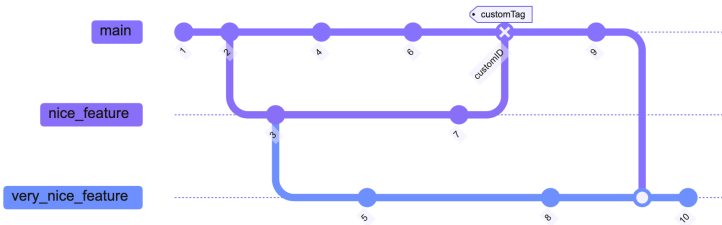
Good practice for branches



- ▶ Keep the main branch clean.
- ▶ Use branches for development.
- ▶ Merge branches to the main branch.
- ▶ Delete branches after merging. (not mandatory)

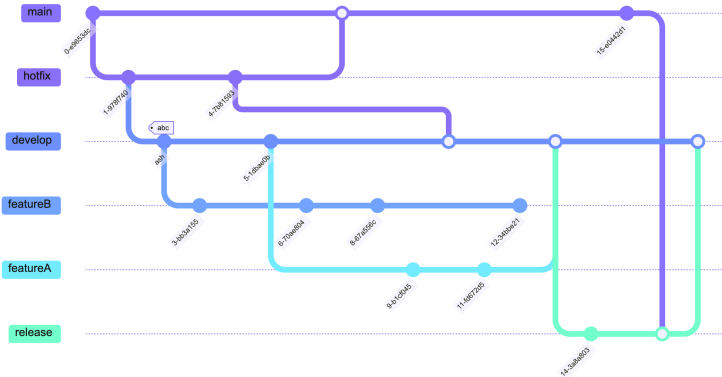
Branches

Example 1



Branches

Example 2



Branches

Take home message



- ▶ Use branches for development.
- ▶ Keep the main branch clean and updated.
- ▶ If collaborate with others, use branches.

Create a branch



- ▶ Create a new branch.
- ▶ Switch to the new branch.
- ▶ Make some changes.
- ▶ Commit the changes.
- ▶ Push the changes to the remote repository.

```
1 git switch -c feature
2 echo "feature" >> README.md
3 git add README.md
4 git commit -m "feat: implement feature 1"
5 git push origin feature
```

Let's see the changes on GitHub and look at the tree both on CLI and GUI.

If you want to switch between existing branches, you can use the following command:

```
1 git switch branch_name
```

When you are done with the development on a branch, you can merge it with the main branch.

To merge branches, you can use the following command:

```
1 git merge branch_name
```

Note: You need to be on the branch where you want to merge the changes (i.e., the main branch).

Merge branches

Is it enough?



Most of the time, the merge is automatic.

However, sometimes you may have conflicts (we will see an example later). Usually git is able to figure out how to merge the changes, but sometimes it needs help.

- ▶ Merge branches.
- ▶ Resolve conflicts.
- ▶ Push the changes.

Merge branches

Typical merge conflict



- ▶ Two branches modify the same line.
- ▶ Git is not able to figure out how to merge the changes.

Usually you will work on different parts of the code (in each branch), hence you will not have conflicts.

Solving in a conflict is generally easy.

- ▶ Open the file with conflicts.
- ▶ Edit the file and resolve the conflicts.
- ▶ Save the file.
- ▶ Add the file.
- ▶ Commit the changes.

But it can be a bit tricky if you are not familiar with the code.
If you have conflict on several files and chunks it can become messy.
There are tools to help you with conflicts. (GUIs, IDEs, etc.)

Advanced usage

- ▶ Advanced usage of Git can be very powerful.
- ▶ But it can also be very complex.
- ▶ It is important to understand the basics before moving to advanced usage.

- ▶ Aliases are shortcuts for Git commands.
- ▶ Can be used to simplify complex commands.
- ▶ Can be used to create new commands.

```
1 git config --global alias.co checkout
2 git config --global alias.br branch
3 git config --global alias.ci commit
4 git config --global alias.st status
5 git config --global alias.adog "log --all --decorate --oneline --graph"
```

- ▶ Force a push to the remote repository.
- ▶ Use it only when you know you are rewriting shared history.
- ▶ It can overwrite other people's work if used carelessly.

```
1 git push --force
```

or you can use the `-force` option.

```
1 git push --force-with-lease
```

Note: Be careful when using the force option, as it can overwrite changes.

Note: A safer way is to use `-force-with-lease` which only pushes if nobody else pushed after your last pull

- ▶ Temporarily save uncommitted changes.
- ▶ Useful when you need to switch branches without creating a commit.
- ▶ Stashes are stored locally and can be applied later.

```
1 git stash
2 git stash pop
```

You can also save a stash with a descriptive message.

```
1 git stash push -m "wip: add login page"
```

Apply the most recent stash without removing it.

```
1 git stash apply
```

- ▶ List all saved stashes.
- ▶ Apply a specific stash by its index.
- ▶ Remove stashes when they are no longer needed.

List the available stashes.

```
1 git stash list
```

Apply a specific stash without removing it.

```
1 git stash apply stash@{1}
```

Remove a stash explicitly.

```
1 git stash drop stash@{1}
```

Apply the latest stash and remove it in one step.

```
1 git stash pop
```

- ▶ Bisect is a binary search tool.
- ▶ Used to find the commit that introduced a bug.
- ▶ Mark a good commit and a bad commit.
- ▶ Git will then perform a binary search to find the commit that introduced the bug.

To start the bisect process, you can use the `bisect start` command.

```
1 git bisect start
```

To mark a commit as bad, you can use the `bisect bad` command.

```
1 git bisect bad
```

To mark a commit as good, you can use the `bisect good` command.

```
1 git bisect good
```

- ▶ Add changes interactively.
- ▶ Select which changes to add to the staging area.
- ▶ Useful when you want to add only specific changes to the staging area.

To add changes interactively, you can use the `-p` option.

```
1 git add -p
```

You can also use the `-i` option to add changes interactively.

```
1 git add -i
```

- ▶ Use history inspection when you want to understand how a repository evolved.
- ▶ It is useful before changing commits, rebasing, or picking a fix from another branch.
- ▶ This is one of the best topics for a live terminal demo.

```
1 git log --graph --decorate --oneline --all
2 git show <commit>
3 git diff <commit1>..<commit2>
```

- ▶ Detached HEAD means you are pointing directly at a commit instead of a branch.
- ▶ It is common when you inspect a tag or an old commit.
- ▶ If you want to keep your changes, create a branch before moving away.

```
1 git switch --detach <commit>
2 git switch -c fix-from-old-state
```

- ▶ Clean removes untracked files and directories.
- ▶ Use the dry-run option first to see what would be deleted.
- ▶ It is handy after experiments, builds, or demo files.

```
1 git clean -n  
2 git clean -fd
```

- ▶ Blame shows which commit last touched each line of a file.
- ▶ It is useful when you want to answer who changed a line and why.
- ▶ Pair it with “git show” to inspect the exact commit afterward.

```
1 git blame <file>  
2 git show <commit>
```

- ▶ Branches can track a remote branch.
- ▶ The upstream branch helps Git understand where to push and pull from.
- ▶ This is a good moment to show how local and remote history differ.

```
1 git branch -vv
2 git branch --set-upstream-to=origin/main
3 git remote show origin
```

- ▶ Fetch downloads remote updates without changing your working branch.
- ▶ Pull combines fetch with merge, or with rebase if you prefer.
- ▶ This is a useful place to compare safe inspection with automatic integration.

```
1 git fetch
2 git pull
3 git pull --rebase
4 git fetch --prune
```

- ▶ A worktree lets you check out another branch in a separate folder.
- ▶ It is useful when you need to compare or fix two branches at the same time.
- ▶ It is a very good command to demo live because the effect is easy to see.

```
1 git worktree add ../repo-hotfix hotfix
2 git worktree list
```

- ▶ Rebase replays your commits on top of another branch.
- ▶ Useful when you want to keep a linear history.
- ▶ Interactive rebase can reorder, edit, squash, and fix up commits.

To move your branch onto a new base, use:

```
1 git rebase <branch>
```

For history cleanup, use interactive rebase:

```
1 git rebase -i <branch>
```

During interactive rebase, common actions are:

- ▶ pick - keep the commit.
- ▶ reword - edit the commit message.
- ▶ squash - combine commits and keep the messages.
- ▶ fixup - combine commits and discard the extra message.

- ▶ Cherry-pick a commit from another branch.
- ▶ Apply the changes made by the commit to the current branch.
- ▶ Useful when you want to apply a specific commit to the current branch.

To cherry-pick a commit, you can use the cherry-pick command.

```
1 git cherry-pick <commit>
```

- ▶ Tags mark important points in history, such as releases.
- ▶ You can list all tags in a repository.
- ▶ Use annotated tags when you want a message and metadata.

```
1 git tag
2 git tag v1.0.0
3 git tag -a v1.0.1 -m "fix: release patch version"
```

Move to an older commit



- ▶ You can move to an older commit to inspect the repository at that exact point in time.
- ▶ If that older state is useful, create a new branch from it so you can keep working safely.
- ▶ This is often used to reproduce a bug, inspect an old release, or start a hotfix from a stable commit.

```
1 git switch --detach <commit>
2 git switch -c hotfix-from-old-commit
```

After the branch is created, you are no longer stuck in detached HEAD and can commit normally.

- ▶ A tag points to a specific commit.
- ▶ Switching to a tag lets you inspect an old release.
- ▶ If you want to keep working from it, create a branch from that tag.

```
1 git switch --detach v1.0.0
2 git switch -c hotfix-v1.0.0 v1.0.0
```

- ▶ Submodules are repositories within a repository.
- ▶ Useful when you want to include external repositories in your project.
- ▶ Can be used to manage dependencies.

To add a submodule to a repository, you can use the `git submodule add` command.

```
1 git submodule add <repository> <path>
```

To populate a submodule, you can use:

```
1 git submodule update --init --recursive
```

- ▶ Clone the main repository with its submodules when possible.
- ▶ Update submodules after cloning if they are missing locally.
- ▶ Remember that a submodule points to a specific commit, not to a branch tip.

```
1 git clone --recurse-submodules <repository>
2 git submodule update --init --recursive
```

A diagram or screenshot of the repository tree helps here, because submodules are easier to understand when you can see the nested repository layout.

- ▶ Hooks are scripts that run before or after a Git command.
- ▶ Can be used to automate tasks.
- ▶ Can be used to enforce rules.

Git has several built-in hooks that you can use.

- ▶ `pre-commit` - Run before a commit.
- ▶ `post-commit` - Run after a commit.
- ▶ `pre-push` - Run before a push.
- ▶ `post-push` - Run after a push.
- ▶ `pre-rebase` - Run before a rebase.
- ▶ `post-rebase` - Run after a rebase.

To create a hook, you need to create a script in the `.git/hooks` directory.



UNIVERSITY
OF TRENTO

Thank you for your attention!